SOBRIÉTÉ DU CODE < CLEAN CODE >

ENSIMAG

17/12/2024

Amine JONAHI

AU PROGRAMME

Sommaire

1. Présentation
2. Clean Code ?
3. Best practices
4. SOLID Principles
5. Dans la pratique
6. SonarQube
7. Workshop

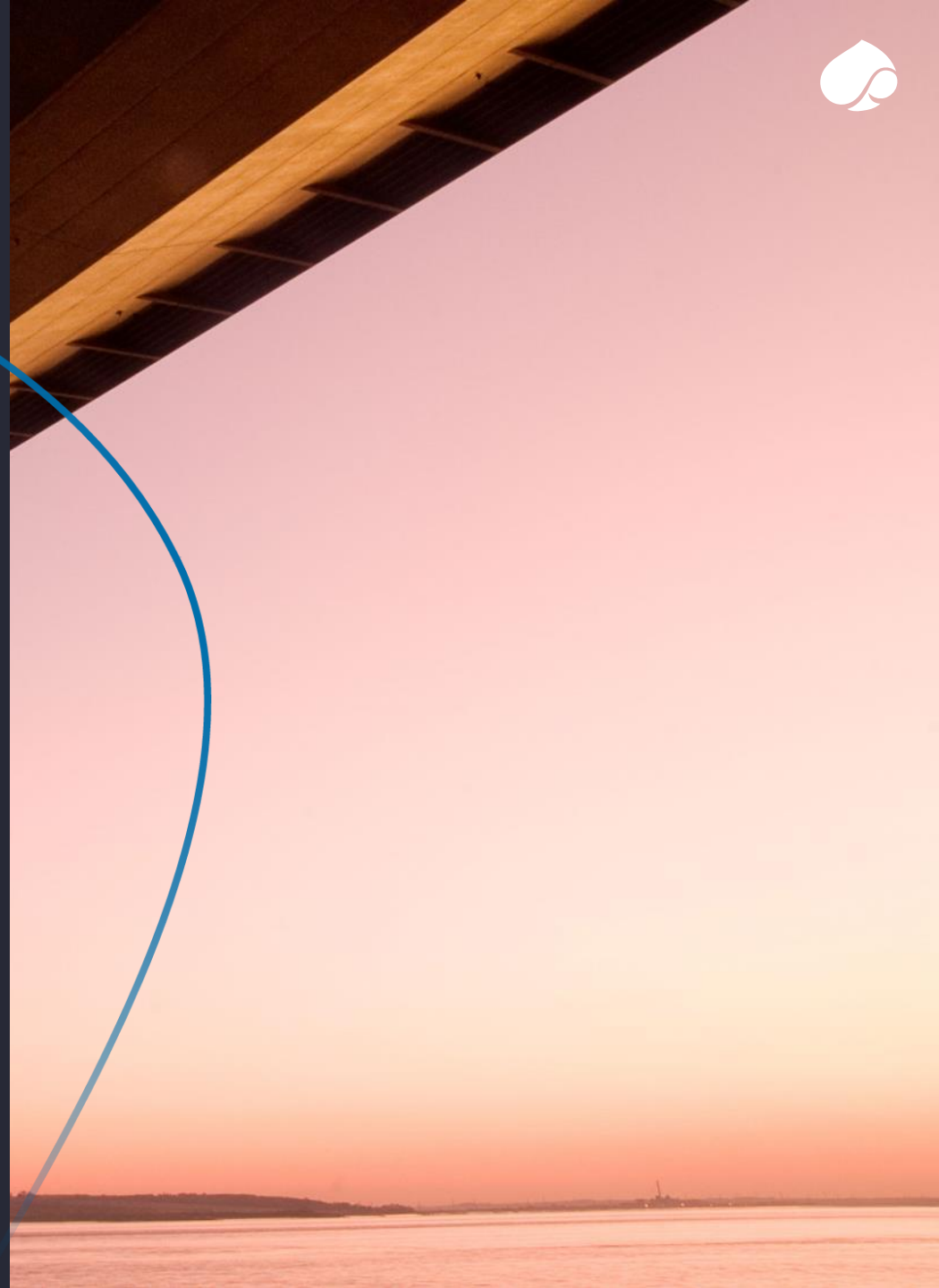
Le support vous sera transmis à la fin du cours

Règles

- On se tutoie
- Mettez vos smartphones de côté, et tout autre canal de communication
- Participez, donnez votre avis, faites part de vos remarques et interrogations (soyons constructifs)
- Respectez les opinions de chacun (bienveillance)

1

PRESENTATION

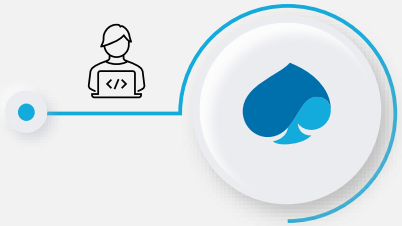


QUI SUIS-JE ?

Amine
JONAHI

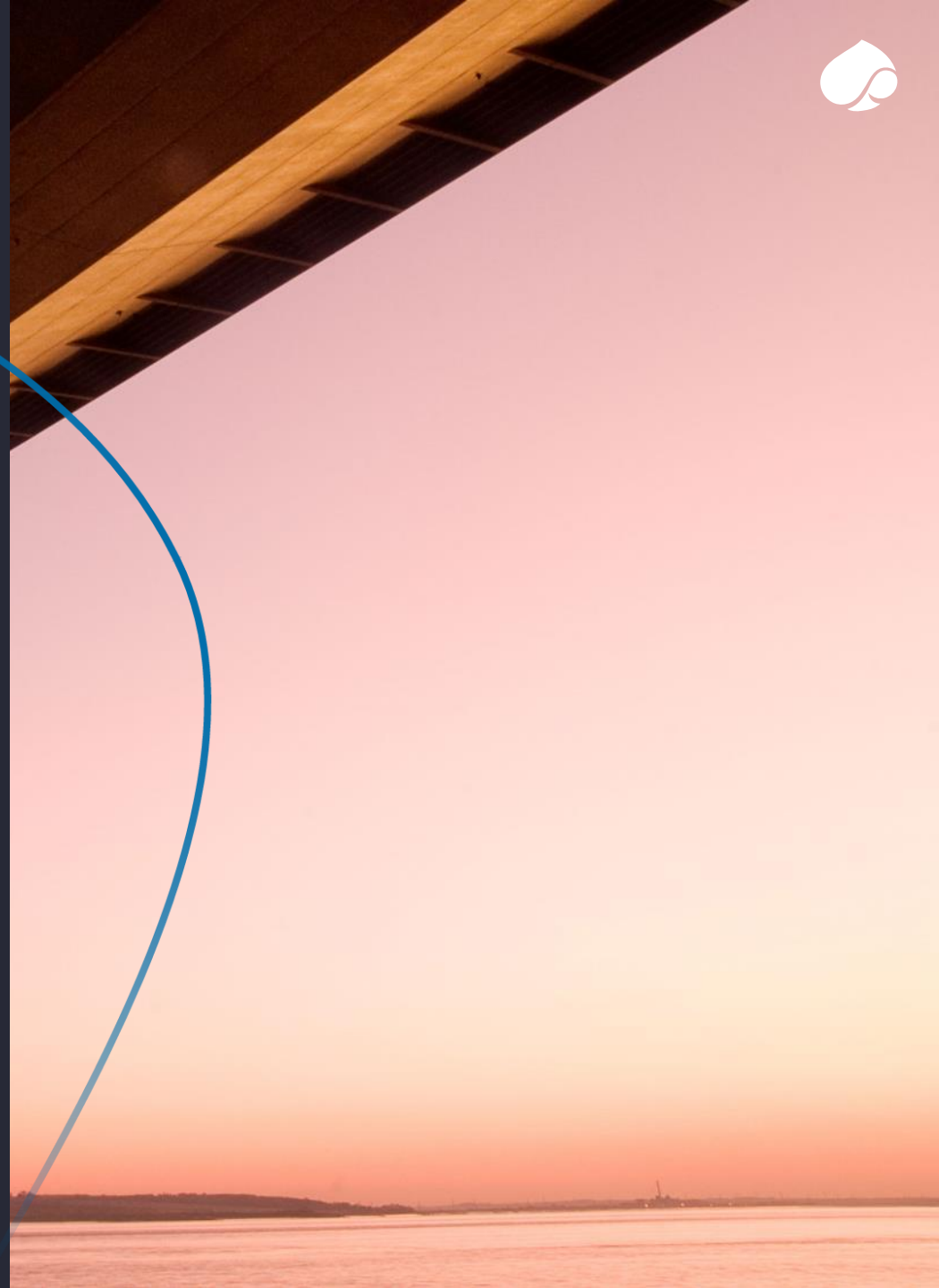
- ***Ingénieur Automatismes, informatique industrielle et systèmes embarqués***
- ***Master Génie logiciel***

Développeur Full Stack
Tech Lead
Admin PL



2

CLEAN CODE



Quiz d'ouverture

60%

du coût d’une application c’est sa maintenance…

Le vrai coût d’une application c’est la maintenance (plus de 60 % du coût de l’application d’après une étude effectuée par le cabinet de conseil Accenture et publiée dans “How Software Maintenance Fees Are Siphoning Away Your IT Budget – and How to Stop It”)

95%

du temps du développeur
c’est la lecture du code…

On passe 10 fois plus de temps à lire le code qu’en écrire (Robert C. Martin dans son ouvrage : « *Clean Code: A Handbook of Agile Software Craftsmanship* »)

UN CLEAN CODE …

C’est un code … propre … c’est-à-dire

- **Clair** et **lisible**
- **Simple** et **concis**
- **Maintenable** et **évolutif** (sans trop coûter)
- **Efficace, Robuste** et **performant**
- **Testé unitairement**
- N’a pas beaucoup de **dépendances**
- Ses **interfaces** sont bien définies
- **Répond à la specification**

Clean Code est un des ***piliers*** de la ***Craftsmanship***

UN CLEAN CODE …

- **Clair** et **Lisible**

```java
public Boolean isAdult(final Integer age) {

    // Bad
    if (age >= 18)
        return true;
    else
        return false;

    // Better
    if (age >= 18) {
        return true;
    } else {
        return false;
    }

    // Good
    if (age >= 18) {
        return true;
    }

    return false;

}
```

UN CLEAN CODE …

- **Robuste et Maintenable**

```java
public String getCountryCode(final String birthplace) {

    // Bad
    if (birthplace == "Italy") {
        return "IT";
    }

    // Better
    if (birthplace.equals("Italy")) {
        return "IT";
    }

    // Good
    if (Objetcs.equals(birthplace, "Italy")) {
        return "IT";
    }

    ...

}
```

Clean Code – Le problème

Le bug de l’an 2000 :

1 Janvier 1970 -> 01-01-70

1 Janvier 2000 -> 01-01-00

Clean Code – La solution

Le bug de l’an 2000 :

1 Janvier 1970 -> 01-01-1970

1 Janvier 2000 -> 01-01-2000

Les leçons à retenir

Le bug de l'an 2000 a été une leçon importante pour la communauté informatique :

- **Importance de la qualité du code:** Un code bien conçu et testé est essentiel pour éviter les erreurs coûteuses.
- **Nécessité de la planification à long terme:** Il est crucial de penser aux conséquences à long terme des choix de conception lors du développement de logiciels.
- **Collaboration internationale:** Les problèmes informatiques peuvent avoir des répercussions mondiales, nécessitant une coopération internationale pour y remédier

Clean Code - Avis de quelques experts

Bjarne Stroustrup

I like my code to be **elegant** and **efficient**. Logic must be **simple** so that bugs can hardly hide. It has **few dependencies** to ease maintenance. My code do **one thing**, and do it well.

Grady Booch

A clean code is **simple** and **direct**. It **can be read** as a good book. It does **not hide the intentions** of the designer. It has **clear abstractions** and **well-defined interfaces**.

Dave Thomas

A clean code **can be read** and **improved** by another developer that the original author. It has **unit tests** and **acceptance tests**. It has **few dependencies** and its **API is clear and minimal**.

(*) Answers copied from "Clean Code" by Robert C. Martin, aka Uncle Bob

Clean Code – Avis de Kent Beck

A Good Code … ✔

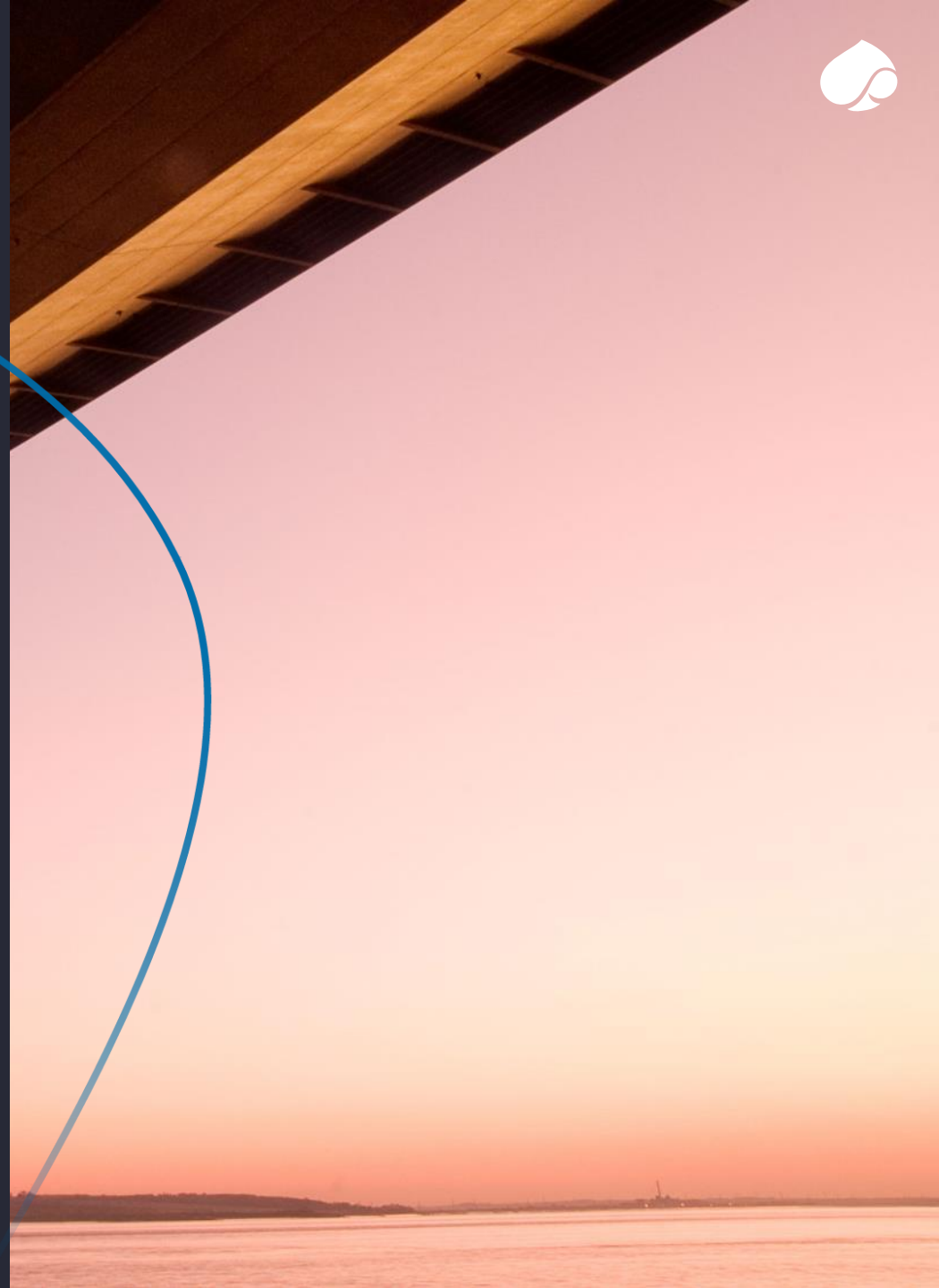
- ✓ **Passes** all the **tests**
- ✓ Is **not redundant**
- ✓ Expresses the **intentions** of the developer
- ✓ **Reduces** the **number** of **classes** and **methods**

Author of XP (eXtreme Programming), co-author of Junit, contributor to the agile manifesto, promotor of TDD

3

CLEAN CODE

BEST PRACTICES



CLEAN CODE – KISS – KEEP IT SIMPLE & STUPID

On trouve aussi

- Keep It Short & Simple
- Keep It Simple & Straightforward

- Garder son code très simple
- Ne pas le complexifier pour la « *beauté* » du code
- S'il est simplement écrit, il est simple à comprendre

*"Everything should be made as simple as possible, but no simpler." -- **Albert Einstein***

*"Simplicity is the ultimate sophistication." -- **Leonardo da Vinci***

CLEAN CODE – YAGNI – YOU AIN'T GONNA NEED IT

Coder **que** ce qui est **immédiatement nécessaire** (efficacité)

Ne **jamais** « prévoir » du code pour le **futur**

➔ But : Gagner du temps et réduire la complexité !

CLEAN CODE – YAGNI – YOU AIN'T GONNA NEED IT

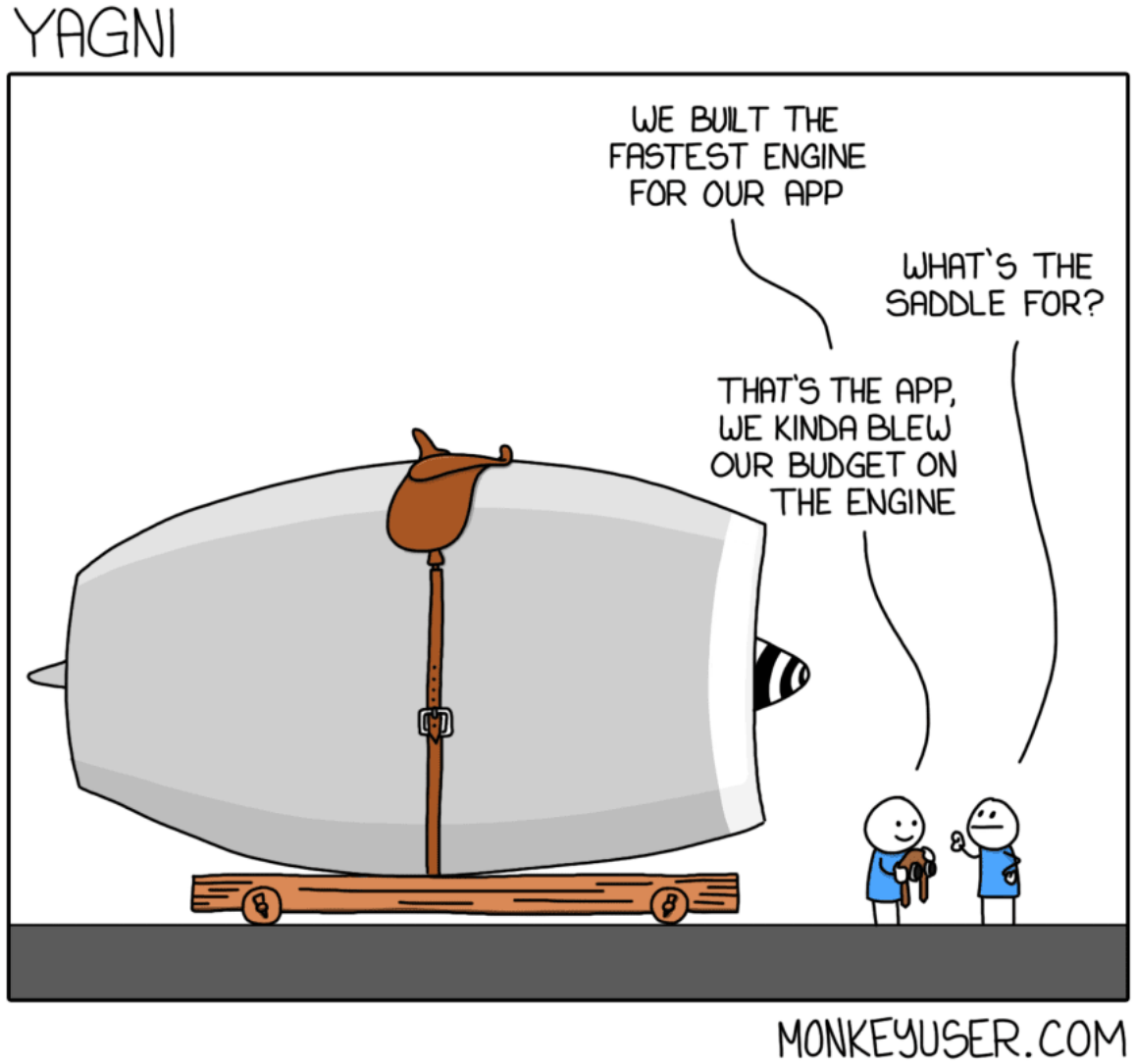
Evaluation de nouveaux frameworks

Décision sur le **design actuel** basée sur des **besoins futurs**

Abstraire les **dépendances externes**

Testing, **sécurité**, **scalabilité** et **exigences** fonctionnelles

CLEAN CODE – YAGNI – YOU AIN'T GONNA NEED IT



CLEAN CODE – DRY – DON’T REPEAT YOURSELF

Eviter au maximum les redondances / duplications de code (copier/coller)

Si un ensemble de code est répété à l’identique plusieurs fois dans le code, alors une fonction est nécessaire.

Once and Only Once (O3) peut être considéré comme un sous-principe de DRY.

La redondance n’est pas à bannir complètement. Parfois elle a du sens.

CLEAN CODE – DRY – DON’T REPEAT YOURSELF

Exemple :

```java
public static int sum (int first, int second){
    if (first > 100 || first < 1 || second > 100 || second < 1){
        throw new IllegalArgumentException("Out of range");
    }

    return  first + second;
}

public static int multiply (int first, int second){
    if (first > 100 || first < 1 || second > 100 || second < 1){
        throw new IllegalArgumentException("Out of range");
    }

    return  first * second;
}
```

```java
public static int sum (int first, int second){
    validateRange(first, second);
    return  first + second;
}

public static int multiply (int first, int second){
    validateRange(first, second);
    return  first * second;
}

private static void validateRange(int first, int second) {
    if (first > 100 || first < 1 || second > 100 || second < 1) {
        throw new IllegalArgumentException("Out of range, must be between 1 and 100");
    }
}
```

CLEAN CODE – SLAP – SINGLE LEVEL OF ABSTRACTION PRINCIPLE

Quoi : Qu’est ce fait le code  Comment : Comment il le fait

Objectif : Cacher la complexité

CLEAN CODE – SLAP – SINGLE LEVEL OF ABSTRACTION PRINCIPLE

Exemple :

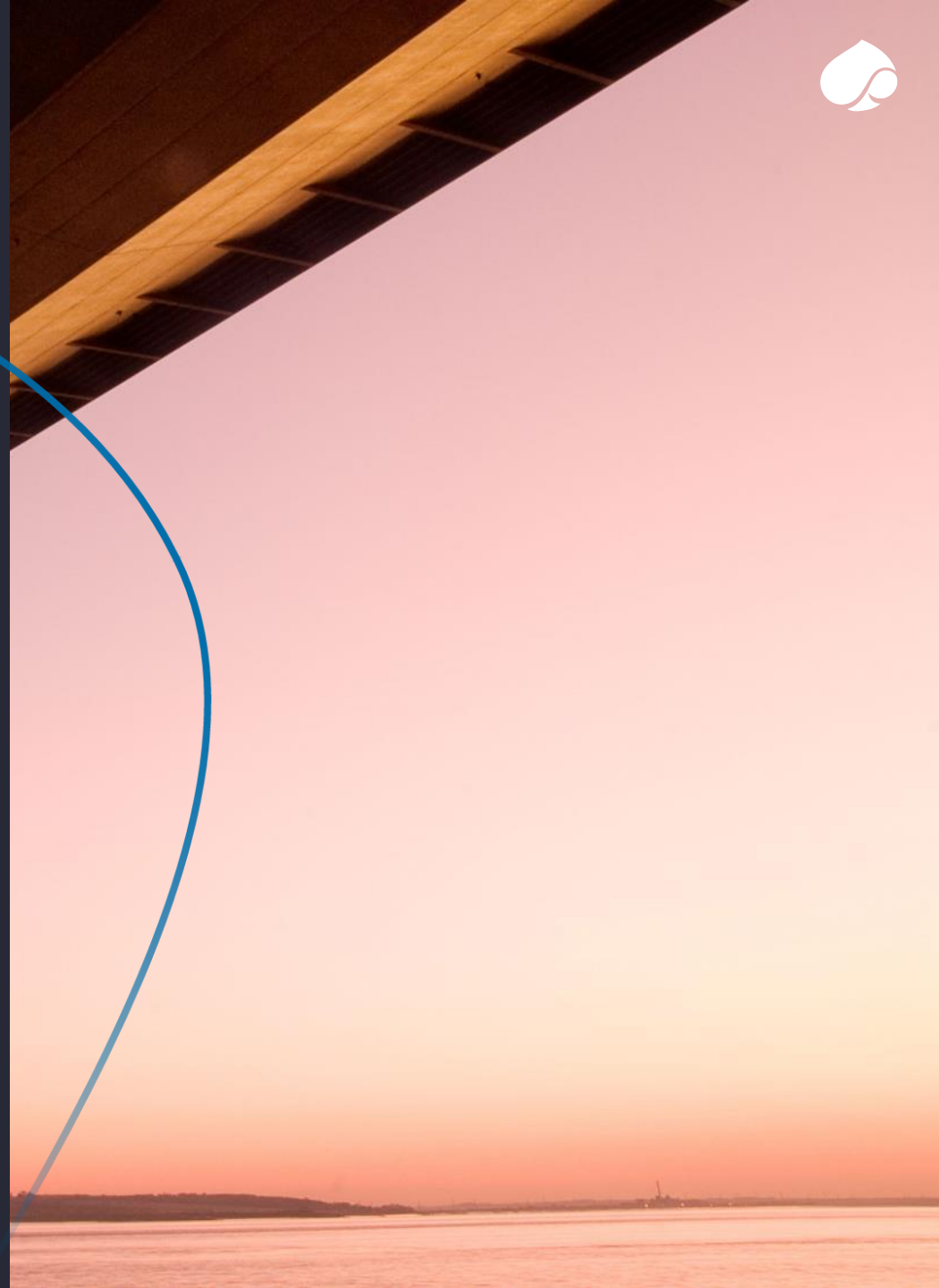
```
class CreateTask(
    private val clock: Clock,
    private val localStorage: LocalStorage,
    private val observers: List<TaskObserver>
) {

    fun invoke(description: String) {
        // normalize description
        val normalizedDescription = if (description.length > MAX_DESCRIPTION_LENGTH)
            description.substring(0, MAX_DESCRIPTION_LENGTH - 1) else
            description

        // create task
        val currentTime = clock.now()
        val initialStatus = Status.NotStarted
        val newTask = Task(normalizedDescription, initialStatus, currentTime)

        // save task
        localStorage.save(newTask)

        // notify
        val observingNotStarting = mutableListOf<TaskObserver>()
        for (i in 0..observers.size) {
            val taskObserver = observers[i]
            if (taskObserver.observedStatus == Status.NotStarted) {
                observingNotStarting.add(taskObserver)
            }
        }
        for (i in 0..observingNotStarting.size) {
            observingNotStarting[i].notify(newTask)
        }
    }
}
```

```
class CreateTask(
    private val clock: Clock,
    private val localStorage: LocalStorage,
    private val observers: List<TaskObserver>
) {

    fun invoke(description: String) {
        val newTask = createNewTask(description)
        localStorage.save(newTask)
        notifyAnyObservers(newTask)
    }

    private fun createNewTask(description: String): Task {
        val normalizedDescription = normalize(description)
        return Task(normalizedDescription, Status.NotStarted, clock.now())
    }

    private fun normalize(description: String): String {
        return if (description.length > MAX_DESCRIPTION_LENGTH)
            description.substring(0, MAX_DESCRIPTION_LENGTH - 1) else
            description
    }

    private fun notifyAnyObservers(newTask: Task) {
        observers
            .filter { taskObserver -> taskObserver.observedStatus == Status.NotStarted }
            .forEach { taskObserver -> taskObserver.notify(newTask) }
    }
}
```

4

CLEAN CODE

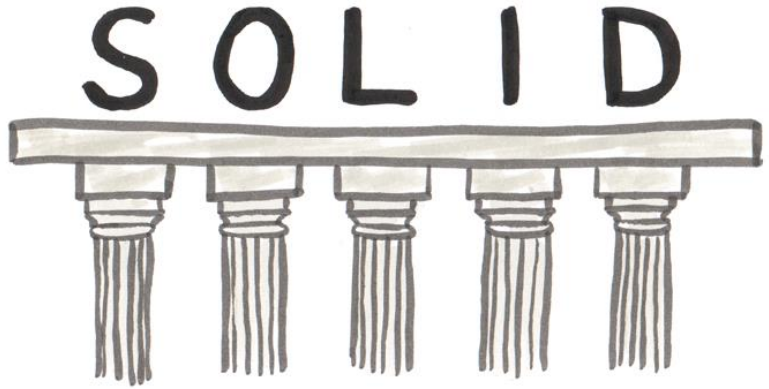
SOLID PRINCIPLES



CLEAN CODE – SOLID

Meta Acronyme composé de 5 principes orientés objet :

- SRP : Single Responsibility Principle
- OCP : Open-Closed Principle
- LSP : Liskov Substitution Principle
- ISP : Interface Segregation Principle
- DIP : Dependency Inversion Principle



CLEAN CODE – SOLID – SRP – SINGLE RESPONSIBILITY PRINCIPLE

Répondre toujours à un besoin à la fois

Exemple :

```
private static void calculateCellsWhereICanSpawnARobot() {
    myCellsWhereICanSpawnRobots = grid.stream()
        .filter(Cell::isOwnedByMe)
        .filter(Cell::isACellWhereICanSpawnARobot)
        .filter(Cell::isNotInARangeOfRecycler)
        .collect(Collectors.toList());
}

private static void calculateDistancesForTheOpCellsThatICanAttack() {
    opCellsThatCanBeAttacked = grid.stream()
        .filter(Cell::isOwnedByOp)
        .filter(Cell::doesNotContainUnits)
        .filter(Cell::isNotInARangeOfRecycler)
        .collect(Collectors.toList());
}

private static void calculateDistancesForTheNewCellsThatICanOwn() {
    xxCellsThatCanBeOwned = grid.stream()
        .filter(Cell::isActive)
        .filter(Cell::isNotOwnedByAnyone)
        .filter(Cell::isNotInARangeOfRecycler)
        .collect(Collectors.toList());
}
```

CLEAN CODE – SOLID – OCP – OPEN-CLOSED PRINCIPLE

Code doit être compatible/ouvert à du code futur sans modifier le code existant

Objectifs :

➔ Architecture du code compatible avec du code futur

➔ Pas de code tant que le besoin n’est pas immédiat et définit

CLEAN CODE – SOLID – OCP – OPEN-CLOSED PRINCIPLE

Exemple :

```java
package com.ezoqc.blog.solid.ocp;

public class UserAgeValidator {
  public boolean isOldEnoughToDrinkAlcohol(int age) {
    return age >= 18;
  }
}
```

```java
package com.ezoqc.blog.solid.ocp;

public class UserAgeValidator {
  public boolean isOldEnoughToDrinkAlcohol(int age, String provinceCode) {
    return provinceCode.equalsIgnoreCase('qc') && age >= 18
      || provinceCode.equalsIgnoreCase('on') && age >= 21
  }
}
```

```java
package com.ezoqc.blog.solid.ocp;

public interface UserAgeValidator {
  boolean isOldEnoughToDrinkAlcohol(int age);
}

public class QuebecAgeValidator implements UserAgeValidator {
  public boolean isOldEnoughToDrinkAlcohol(int age) {
    return age >= 18;
  }
}

public class OntarioAgeValidator implements UserAgeValidator {
  public boolean isOldEnoughToDrinkAlcohol(int age) {
    return age >= 21;
  }
}
```

CLEAN CODE – SOLID – LSP – LISKOV SUBSTITUTION PRINCIPLE

Nom basé par sa créatrice Barbara Liskov

Pouvoir substituer dans une classe les propriétés de celles-ci

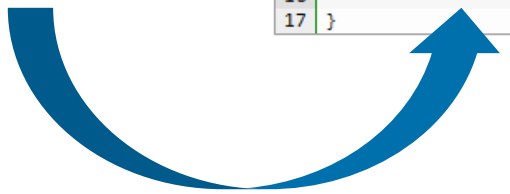
Principe de programmation orienté objet :

- Classes
- Interfaces
- Types et sous-types

CLEAN CODE – SOLID – LSP – LISKOV SUBSTITUTION PRINCIPLE

Exemple :

```java
public class User {
  private String emailAddress;

  public String getEmailAddress() {
    return this.emailAddress;
  }
}

public class AnonymousUser extends User {
  public String getEmailAddress() {
    throw new RuntimeException("Anonymous users don't have an email address.");
  }
}
```



```java
package com.ezoqc.blog.solid.lsp;

public abstract class User {

}

public class RegisteredUser extends User{
  private String emailAddress;

  public String getEmailAddress() {
    return this.emailAddress;
  }
}

public class AnonymousUser extends User {

}
```

CLEAN CODE – SOLID – ISP – INTERFACE SEGREGATION PRINCIPLE

Un client utilisant une interface ne doit pas embarquer obligatoirement des méthodes inutiles

Objectif :

➔ Découper fonctionnellement les interfaces (et les méthodes qu'elles ont) selon la pertinence d'emploi par les clients

CLEAN CODE – SOLID – ISP – INTERFACE SEGREGATION PRINCIPLE

Exemple :

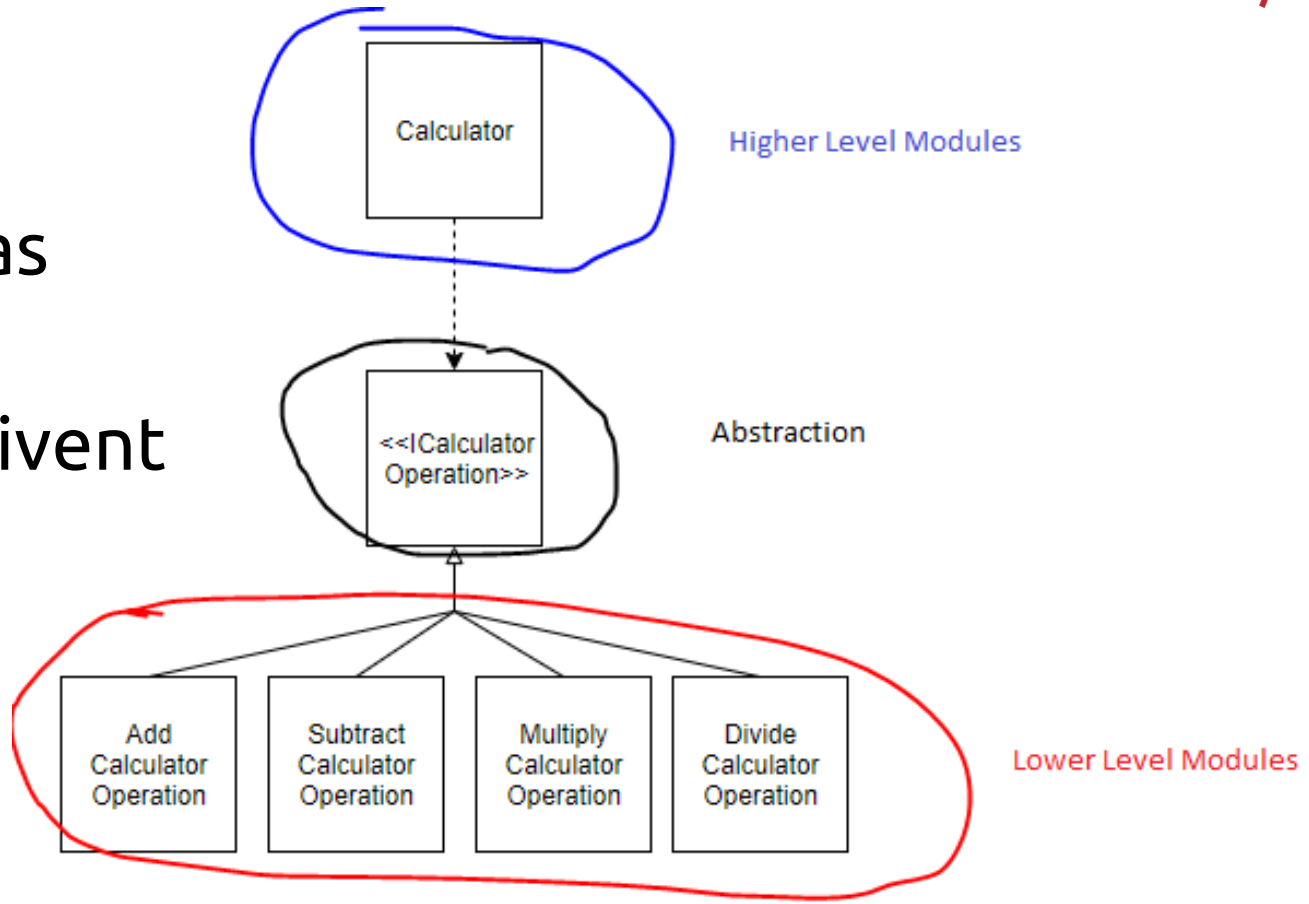
```java
interface Animal {

    void eat();

    void fly();

    void swim();
}

class Dog implements Animal {

    @Override
    public void eat() {
        System.out.println("Dog eats !");
    }

    @Override
    public void fly() {
        // Throws ERROR - dogs cannot fly
    }

    @Override
    public void swim() {
        // Throws ERROR - dogs cannot swim
    }
}

class Eagle implements Animal {

    @Override
    public void eat() {
        System.out.println("Eagle eats !");
    }

    @Override
    public void fly() {
        System.out.println("Eagle flies !");
    }

    @Override
    public void swim() {
        // Throws ERROR - dogs cannot swim
    }
}
```

```java
interface Animal {

    void eat();
}

interface AnimalWhichFlies extends Animal {

    void fly();
}

interface AnimalWhichSwims extends Animal {

    void swim();
}

class Dog implements Animal {

    @Override
    public void eat() {
        System.out.println("Dog eats !");
    }
}

class Eagle implements AnimalWhichFlies {

    @Override
    public void eat() {
        System.out.println("Eagle eats !");
    }

    @Override
    public void fly() {
        System.out.println("Eagle flies !");
    }
}
```

CLEAN CODE – SOLID – DIP – DEPENDENCY INVERSION PRINCIPLE

Les abstractions ne doivent pas dépendre des détails

Mais, plutôt, les détails qui doivent dépendre des abstractions



CLEAN CODE – SOLID – DIP – DEPENDENCY INVERSION PRINCIPLE

Exemple :

```
const dbConn = //....

export function saveUser(user) {
    return dbConn.save(user)
}
```

```
export function saveUser(user, dbConn) {
    return dbConn.save(user)
}
```

FAIRE DU CLEAN CODE : COMMENT Y PARVENIR ?

- Best practices

  - KISS (Keep It Simple Stupid)
  - YAGNI (You Ain't gonna need it)
  - DRY (Don’t Repeat Yourself)
  - SLAP (Single Level of Abstraction Principle)

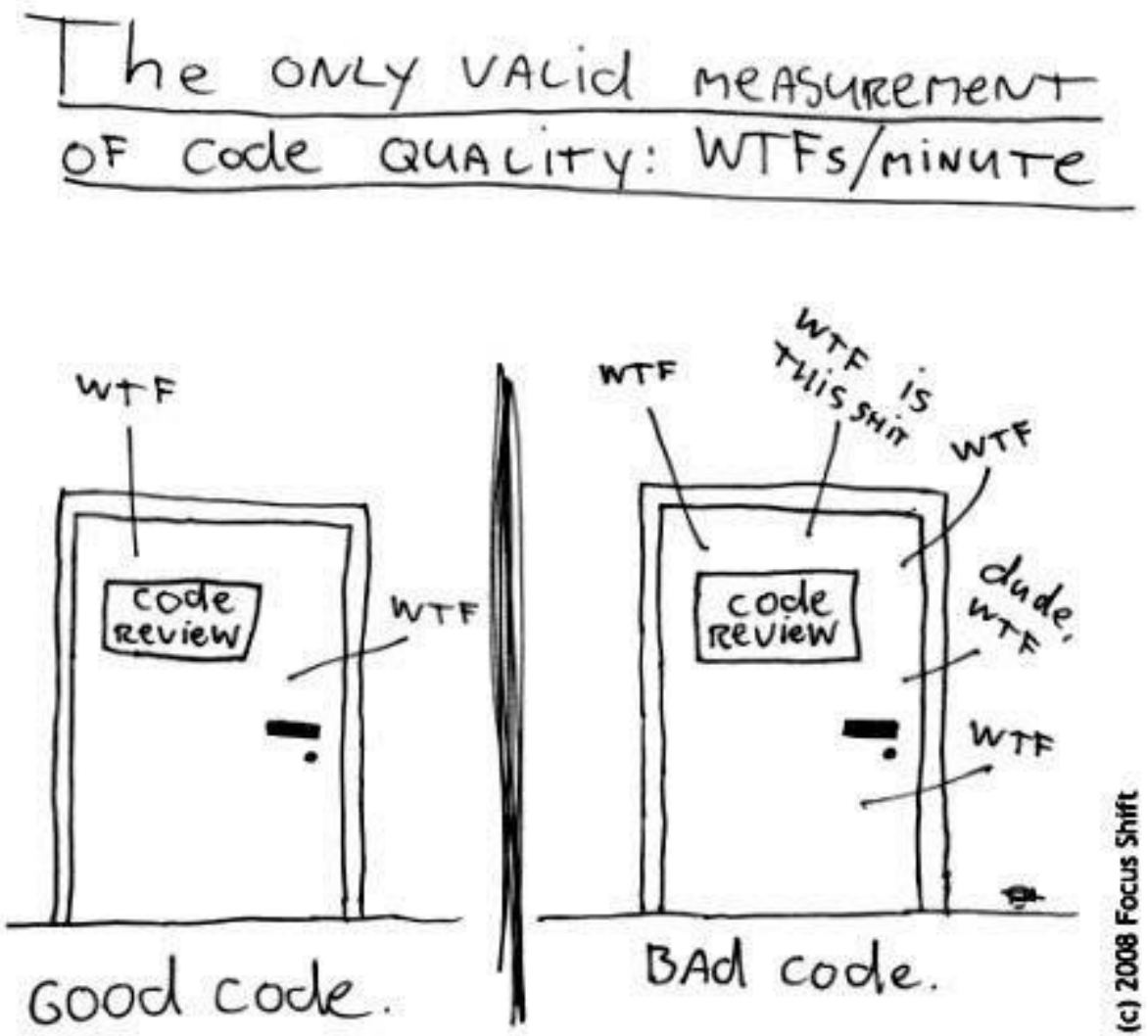
- Des méthodologies

  - SOLID : 5 principes de design

- Bon sens

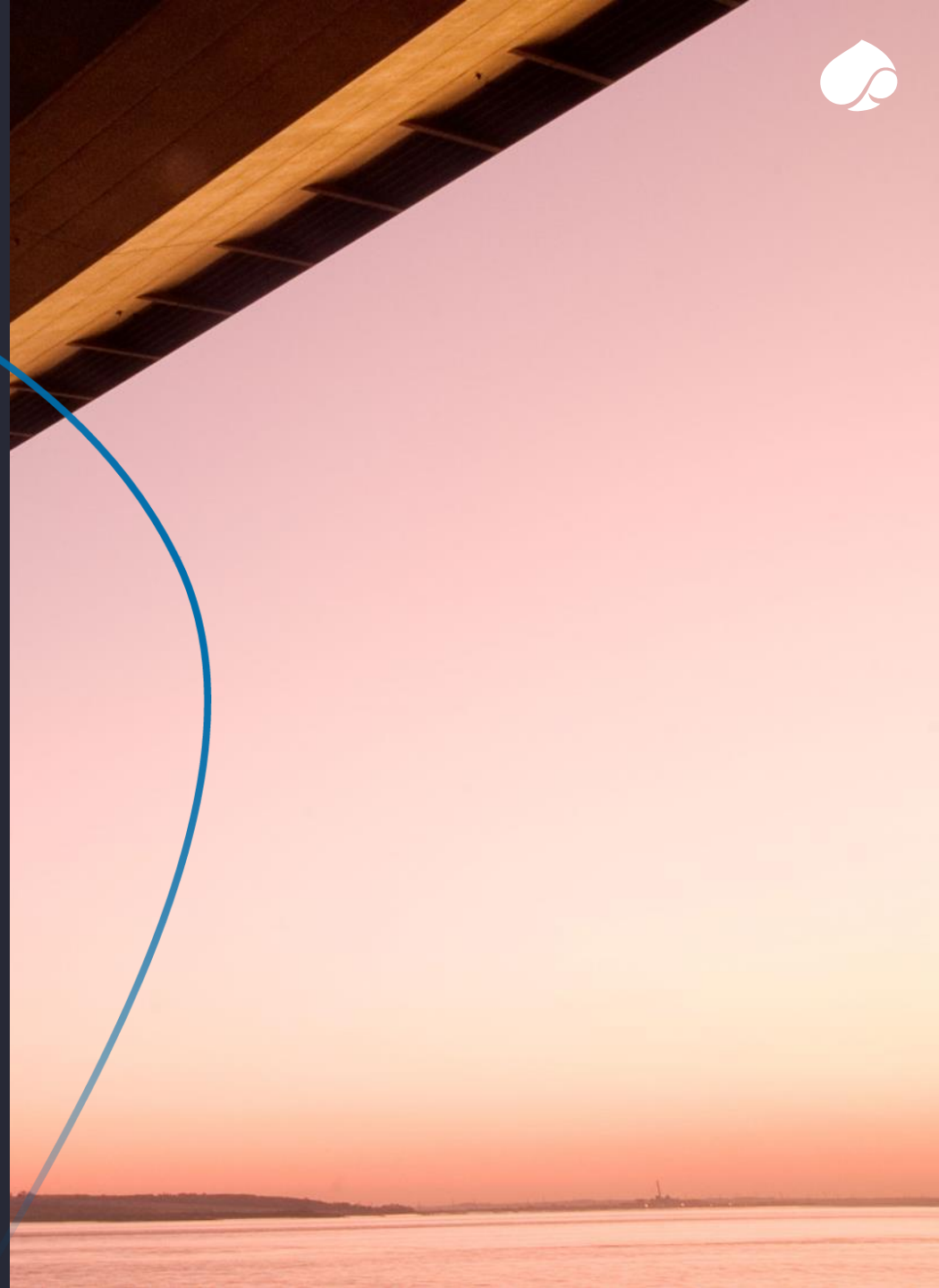
  - Cause racine (lors de la résolution des problèmes trouver toujours la cause racine)
  - Boy Scout Rule (laisser le camp plus propre que l’état dans lequel vous l’avez trouvé)
  - Hollywood Principle (“Don't Call Us, We'll Call You”, le code répond à des évènements externes)
  - Tell don’t ask (plutôt que de demander à un objet les données et agir dessus, déplacer la logique métier dedans)
  - Bien nommer son code (champs, méthodes, variables, paramètres, interfaces, classes, annotations, etc…)
  - Bien gérer les erreurs (exceptions au lieu des codes retour)
  - Commenter le code (juste ce qu’il faut ☺)
  - Ecrire les tests

FAIRE DU CLEAN CODE : COMMENT Y PARVENIR ?



5

DANS LA PRATIQUE



Capgemini

UN BON DEV ?

UN BON DEV

PRAGMATIQUE

SAIT ARITCULER LES CONCEPTS

EXPRIMER SA PENSÉE

CAPACITÉ DE RÉFLEXION

A DU BON SENS

S'ADAPTE À LA SITUATION

Capgemini

DANS LA PRATIQUE …

Code Nouveau suivant les bons principes

- Clean Code : écrire du bon code
- Clean Design : bien concevoir
- Clean Architecture : bien choisir sa(ses) structure(s) logicielle(s)

➔ **Bannir la sur-ingénierie et la sur-spécification**

Code Legacy : intervention suivant Boy Scout Rule

- On laisse le code plus propre que son état initial

➔ **Rachat de la Dette Technique**

DANS LA PRATIQUE …

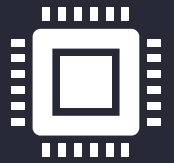
100%

des développeurs ont l’obligation de faire du

- ✓ Clean Code
- ✓ Clean Design
- ✓ Green Code

CONCLUSION

Plus rapide pour corriger des bugs



Code plus performant

RÉFÉRENCES

Books

Clean Code: A Handbook of Agile Software Craftsmanship - Robert C-Martin

The Pragmatic Programmer — David Thomas and Andrew Hunt

Software Architect’s Handbook — Joseph Ingeno

Extreme Programming Installed — Ron Jeffries, Ann Anderson, Chet Hendrickson

Agile Software Development, Principles, Patterns, and Practices

Green pattern manuel d’éco-conception des logiciels

https://www.lulu.com/shop/green-code-lab/green-patterns-manuel-d%C3%A9co-conception-des-logiciels/ebook/product-18848920.html?page=1&pageSize=4

Website

https://refactoring.guru/fr/design-patterns
https://www.pluralsight.com/courses/principles-oo-design
https://en.wikipedia.org/wiki/Single-responsibility_principle
https://blog.adimeo.com/forum-php-2019-clean-architecture
https://deviq.com/
https://blog.ippon.fr/2018/05/14/mon-avis-sur-le-livre-clean-architecture-par-robert-c-martin/
http://principles-wiki.net/principles:start
https://thevaluable.dev/dry-principle-cost-benefit-example/
https://dev.to/codemouse92/clean-dry-solid-spaghetti-1lgm
https://martinfowler.com/bliki/TellDontAsk.html
https://medium.com/javarevisited/slap-that-ugly-code-6ec276d3a4bc
https://le0nidas.gr/2020/12/26/slap-single-level-of-abstraction-principle/
http://blog.ezoqc.com/5-exemples-faciles-pour-comprendre-les-principes-solid/
https://www.javatpoint.com/solid-principles-java

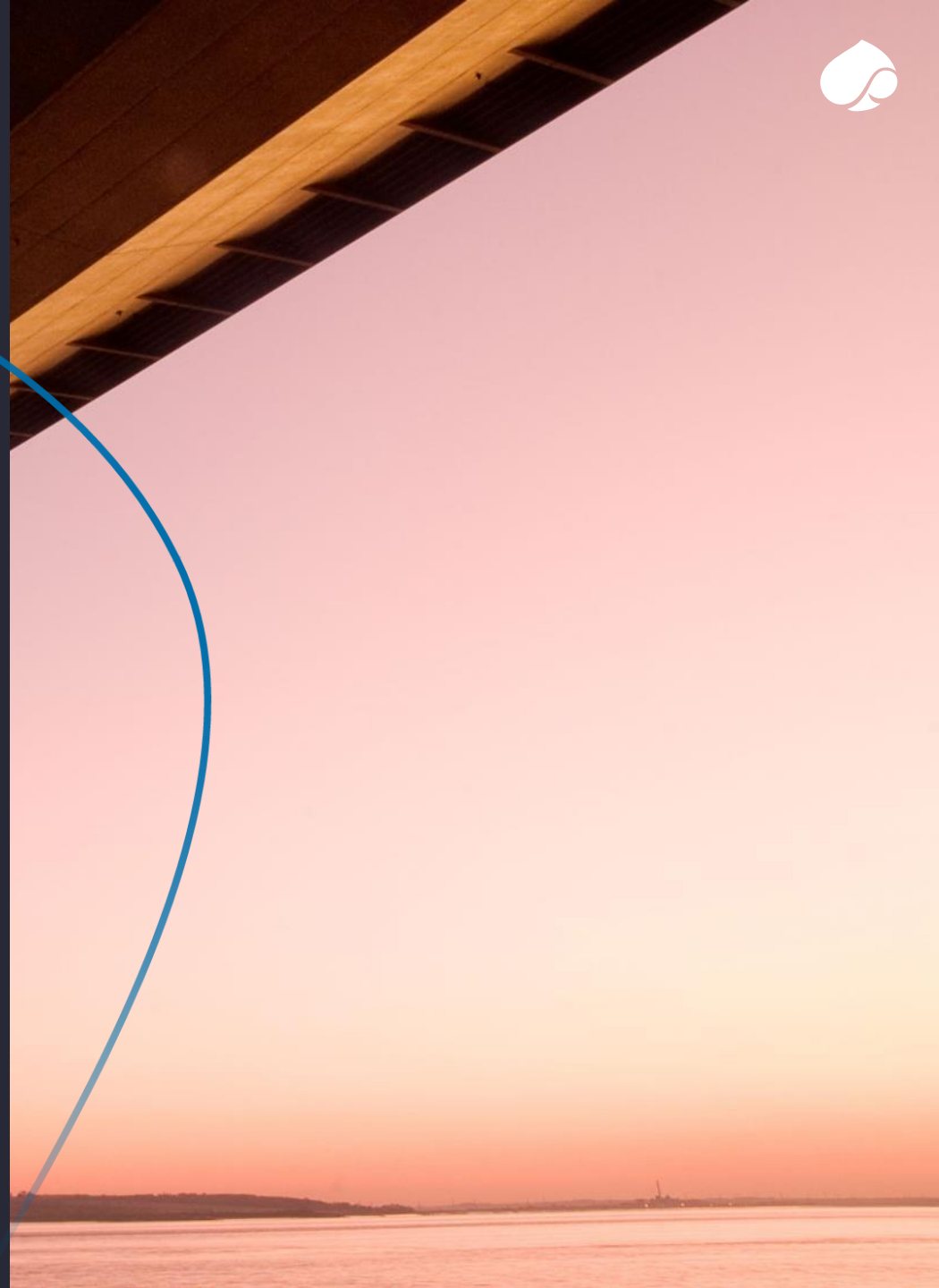
Videos

Oncle Bob

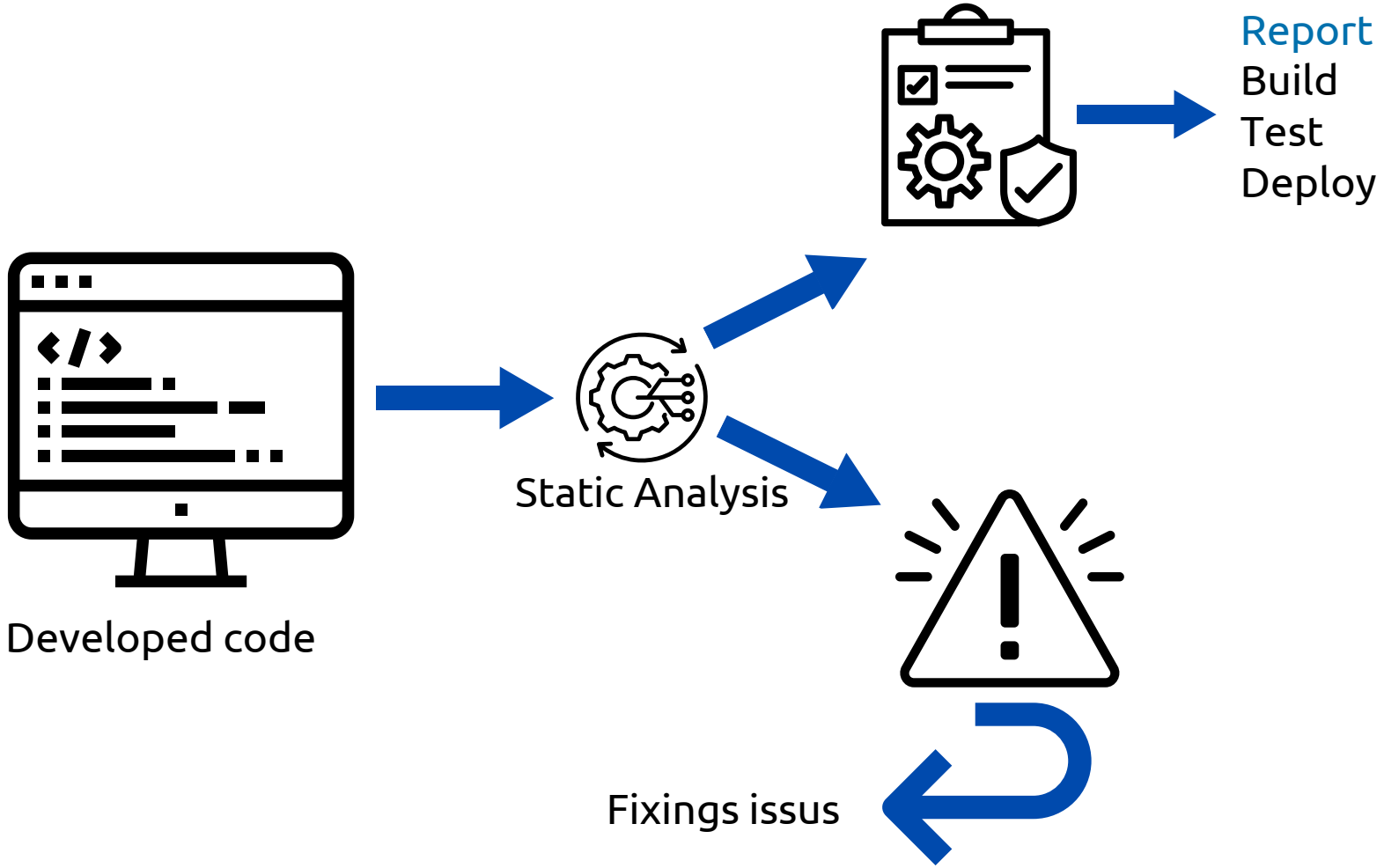
https://www.youtube.com/watch?v=7EmboKQH8lM

6

SONARQUBE



SONARQUBE – ANALYSE STATIQUE DU CODE

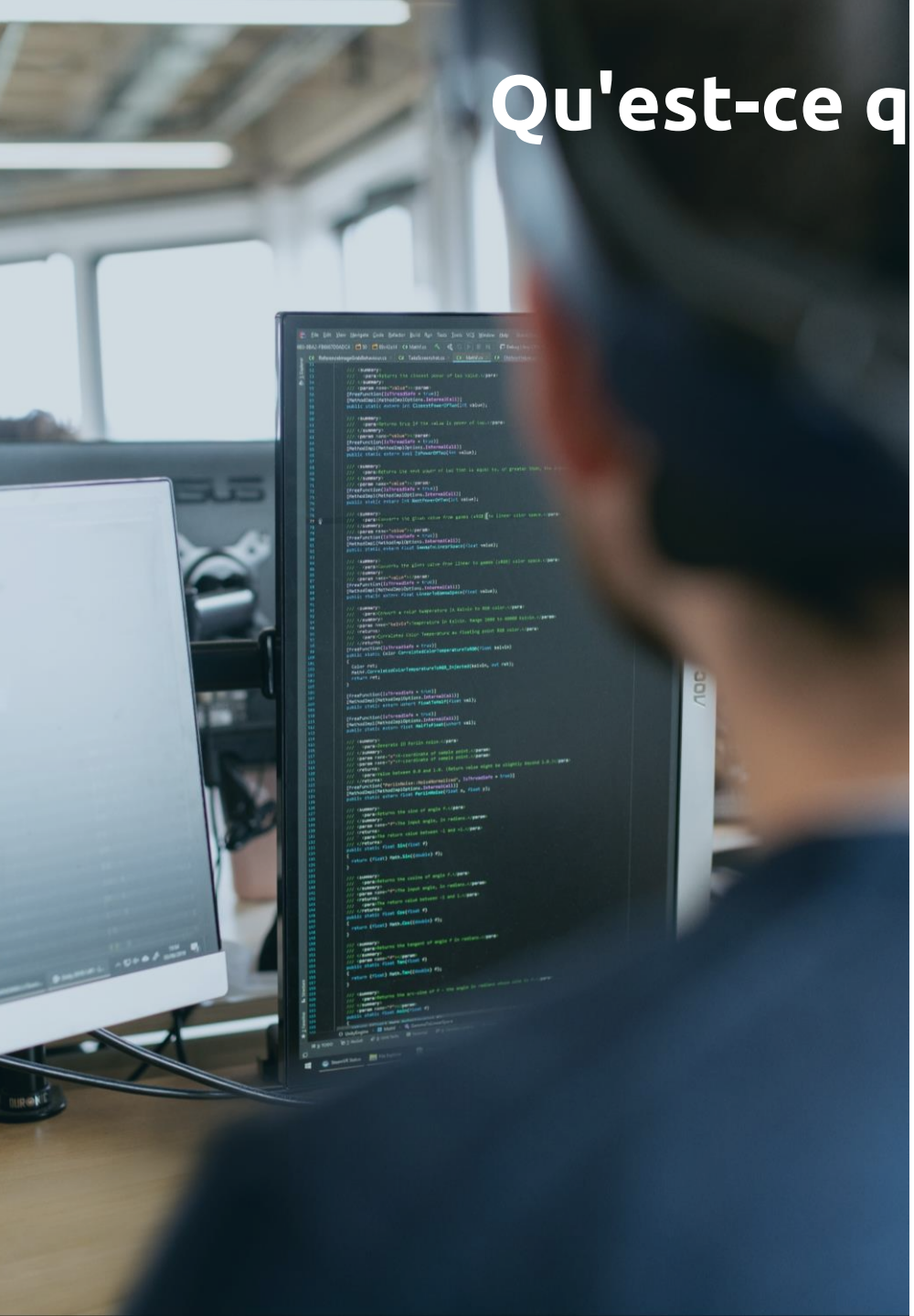


EXAMPLE – ANALYSE STATIQUE DU CODE

```java
public class CodeMortExample {
    public static void main(String[] args) {
        boolean condition = false;

        if (condition) {
            System.out.println("Ce message ne s'affichera jamais");
        } else {
            System.out.println("Ce message s'affichera toujours");
        }
        int x = 10;
        System.out.println(x);
    }
}
```

Ce bloc de code ne sera jamais exécuté
car 'condition' est toujours false

Ce bloc de code est également inaccessible
car le 'return' précédent interrompt l'exécution

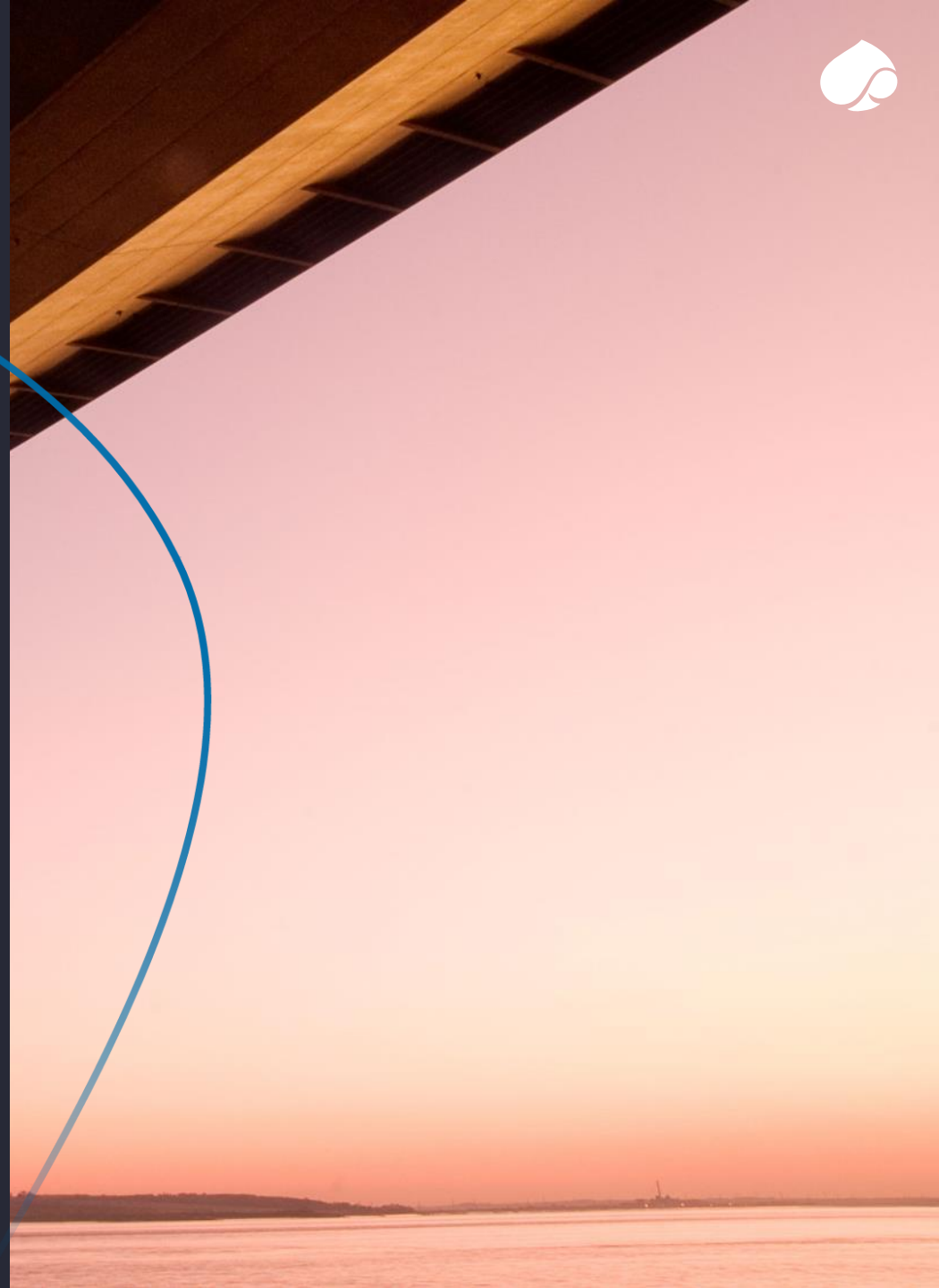


Qu'est-ce que SonarQube ?

- Un outil d'analyse de code statique.
- Permet d'évaluer la qualité du code.
- Identifie les bugs potentiels, les vulnérabilités, les duplications, etc.
- Améliore la maintenabilité et la fiabilité du code.

7

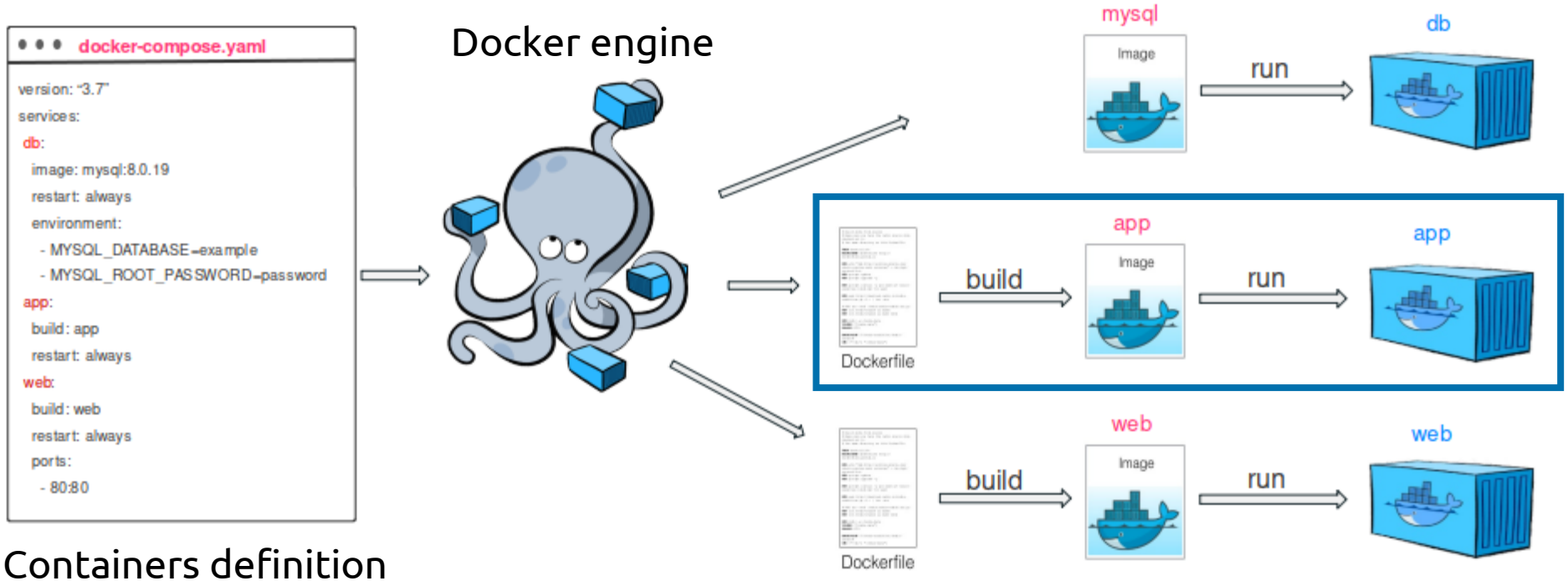
WORKSHOP



WORKSHOP SNARQUBE – PREREQUIS

- Avoir un os Linux ou Windows avec WSL
- Installer Docker & Docker compose: https://docs.docker.com/get-started/get-docker/
- Télécharger les sources du cours: https://gitlab.com/ensimag-sc-1724

WORKSHOP SNARQUBE – DOCKER

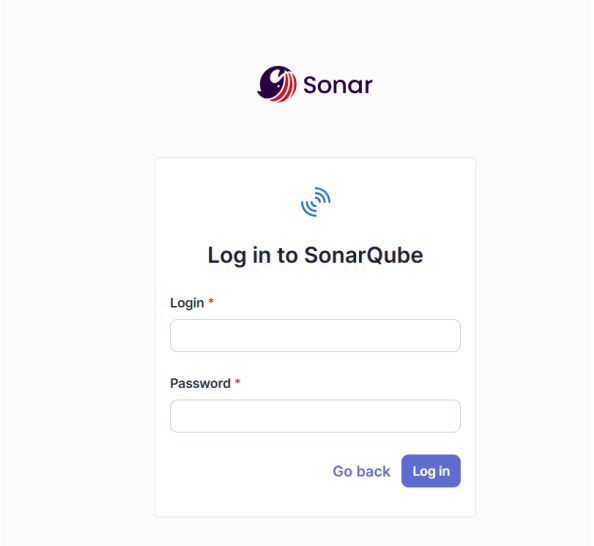


Workshop SnarQube – Installation

- Décompresser l’archive fournie dans le cours sur la machine contenant docker :
  unzip –o sq-community-docker.zip
- Se déplacer dans le repertoire :
  cd “sq-community-docker”
- Ouvrir un terminal et exécuter:
  docker compose up

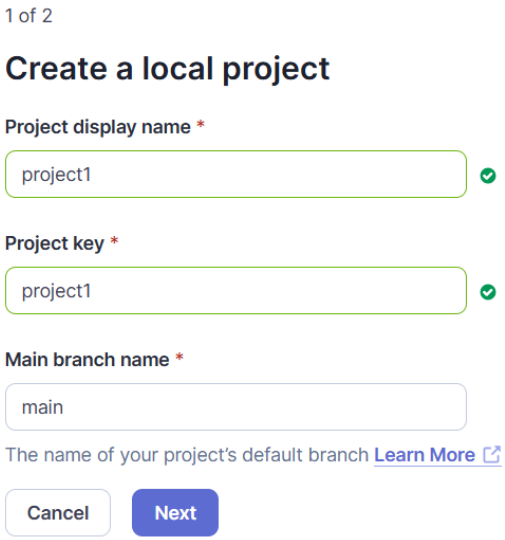
Workshop SnarQube – Lancement

- Visiter la page :
  http://localhost:9000/
- S’authentifier avec :
  admin/admin
- Définir un nouveau mot de passe
- S’authentifier à nouveau avec le nouveau mot de passe

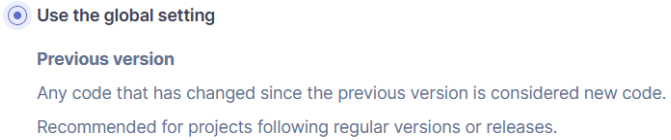


WORKSHOP SNARQUBE – CONFIGURATION

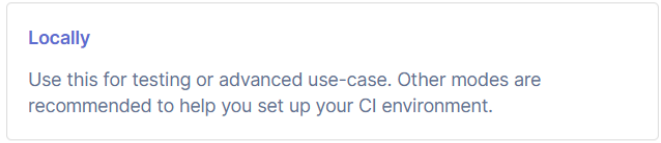
1



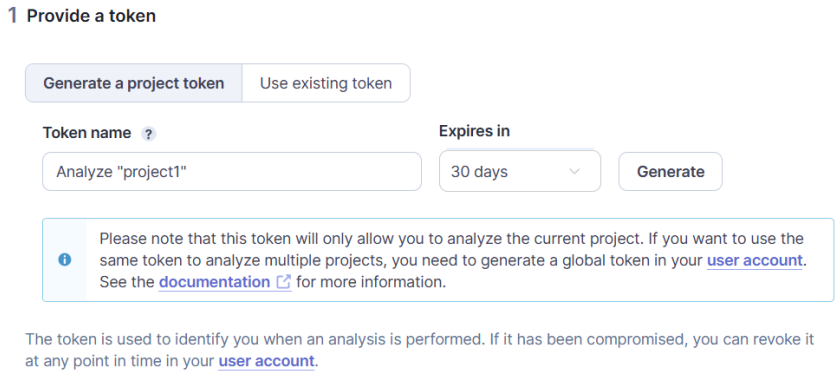
2



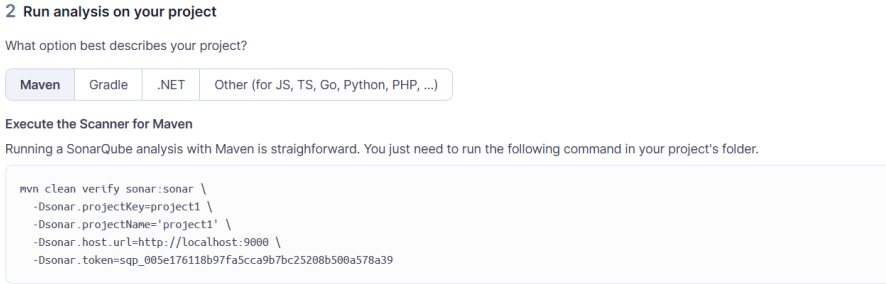
3



4



5

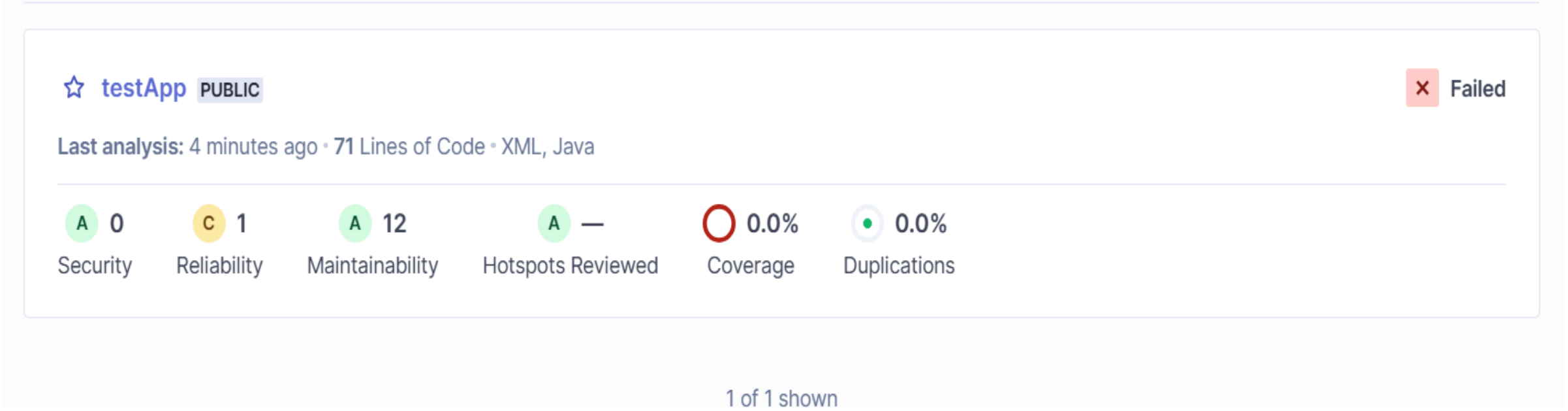


WORKSHOP SNARQUBE – RAPPORT

Lancer l’analyse du code sur votre machine local

mvn clean verify sonar:sonar -Dsonar.projectKey=testApp -Dsonar.projectName='testApp’ …

Visualiser le rapport du scan de votre projet sur le dashboard sonarQube



Capgemini

GET THE FUTURE YOU WANT

capgemini.com